

CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

August 21, 2025

Lecture 2

Basic Text Processing

Regular Expression

- Part of every text processing task
 - Not a general NLP solution (for that we use large NLP systems we will see in later lectures)
 - But very useful as part of those systems (e.g., for pre-processing or text formatting)
- Necessary for data analysis of text data
- A widely used tool in industry and academics

Regular expressions

A formal language for specifying text strings

How can we search for mentions of these cute animals in text?

- woodchuck
- woodchucks
- Woodchuck
- Woodchucks
- Groundhog
- groundhogs



Regular Expressions: Disjunctions

Letters inside square brackets []

Pattern	Matches
<code>[wW]oodchuck</code>	Woodchuck, woodchuck
<code>[1234567890]</code>	Any one digit

Ranges using the dash `[A-Z]`

Pattern	Matches	
<code>[A-Z]</code>	An upper case letter	<u>D</u> renched Blossoms
<code>[a-z]</code>	A lower case letter	<u>my</u> beans were impatient
<code>[0-9]</code>	A single digit	Chapter <u>1</u> : Down the Rabbit Hole

Regular Expression

Wildcards, optionality, repetition: . ? * +

Pattern	Matches	Examples
<code>beg.n</code>	Any char	<u>begin</u> <u>begun</u> <u>beg3n</u> <u>beg n</u>
<code>woodchucks?</code>	Optional s	<u>woodchuck</u> <u>woodchucks</u>
<code>to*</code>	0 or more of previous char	<u>t</u> <u>to</u> <u>too</u> <u>tooo</u>
<code>to+</code>	1 or more of previous char	<u>to</u> <u>too</u> <u>tooo</u> <u>toooo</u>



Stephen C Kleene
Kleene *, Kleene +

Regular Expressions: Negation in Disjunction

Carat as first character in [] negates the list

- Note: Carat means negation only when it's first in []
- Special characters (., *, +, ?) lose their special meaning inside []

Pattern	Matches	Examples
[^A-Z]	Not an upper case letter	O <u>y</u> fn pripetchik
[^Ss]	Neither 'S' nor 's'	<u>I</u> have no exquisite reason"
[^.]	Not a period	<u>O</u> ur resident Djinn
[e^]	Either e or ^	Look up <u>^</u> now

Regular Expressions: Convenient aliases

Pattern	Expansion	Matches	Examples
<code>\d</code>	<code>[0-9]</code>	Any digit	Fahreheit <u>4</u> 51
<code>\D</code>	<code>[^0-9]</code>	Any non-digit	<u>B</u> lue Moon
<code>\w</code>	<code>[a-zA-Z0-9_]</code>	Any alphanumeric or _	<u>D</u> aiyu
<code>\W</code>	<code>[^\w]</code>	Not alphanumeric or _	Look <u>!</u>
<code>\s</code>	<code>[\r\t\n\f]</code>	Whitespace (space, tab)	Look <u>_</u> up
<code>\S</code>	<code>[^\s]</code>	Not whitespace	<u>L</u> ook up

Delta can sue CrowdStrike over computer outage that caused 7,000 canceled flights

By Jonathan Stempel

May 19, 2025 3:12 PM EDT · Updated May 19, 2025



Aa



The iterative process of writing regex's

Find me all instances of the word “the” in a text.

`the`

Misses capitalized examples

`[tT]he`

Incorrectly returns other or Theology

`\W[tT]he\W`

False positives and false negatives

The process we just went through was based on fixing two kinds of errors:

1. Not matching things that we should have matched
(The)

False negatives

2. Matching strings that we should not have matched
(there, then, other)

False positives

Regular expressions play a surprisingly large role

Widely used in both academics and industry

1. Part of most text processing tasks, even for big neural language model pipelines
 - including text formatting and pre-processing
2. Very useful for data analysis of any text data

How many words in a sentence?

"I do uh main- mainly business data processing"

- Fragments, filled pauses

"Seuss's **cat** in the hat is different from other **cats**!"

- **Lemma**: same stem, part of speech, rough word sense
 - **cat** and **cats** = same lemma
- **Wordform**: the full inflected surface form
 - **cat** and **cats** = different wordforms

How many words in a sentence?

they lay back on the San Francisco grass and looked at the stars
and their

Type: an element of the vocabulary.

Token: an instance of that type in running text.

How many?

- 15 tokens (or 14)
- 13 types (or 12) (or 11?)

How many words in a corpus?

N = number of tokens

V = vocabulary = set of types, **|V|** is size of vocabulary

Heaps Law = Herdan's Law = $|V| = kN^\beta$ where often $.67 < \beta < .75$

i.e., vocabulary size grows with $>$ square root of the number of word tokens

	Tokens = N	Types = V
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
COCA	440 million	2 million
Google N-grams	1 trillion	13+ million

Corpora vary along dimension like

- **Language:** 7097 languages in the world
- **Variety**, like African American Language varieties.
 - AAE Twitter posts might include forms like "*iont*" (*I don't*)
- **Code switching**, e.g., Spanish/English, Hindi/English:
 - S/E: Por primera vez veo a @username actually being hateful! It was beautiful:)
[For the first time I get to see @username actually being hateful! it was beautiful:]
 - H/E: dost tha or ra- hega ... dont worry ... but dherya rakhe
["he was and will remain a friend ... don't worry ... but have faith"]
- **Genre:** newswire, fiction, scientific articles, Wikipedia
- **Author Demographics:** writer's age, gender, ethnicity, SES

Text Normalization

Every NLP task requires text normalization:

1. Tokenizing (segmenting) words
2. Normalizing word formats
3. Segmenting sentences

Space-based tokenization

A very simple way to tokenize

- For languages that use space characters between words
 - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
- Segment off a token between instances of spaces

Unix tools for space-based tokenization

- The "tr" command
- Inspired by Ken Church's UNIX for Poets
- Given a text file, output the word tokens and their frequencies

Words and Corpus

```
tr -sc 'A-Za-z' '\n' < shakes.txt | head
```

THE

SONNETS

by

William

Shakespeare

From

fairest

creatures

We

...

Issues in Tokenization

Can't just blindly remove punctuation:

- m.p.h., Ph.D., AT&T, cap'n
- prices (\$45.55)
- dates (01/02/06)
- URLs (<http://www.stanford.edu>)
- hashtags ([#nlproc](#))
- email addresses (someone@cs.colorado.edu)

Clitic: a word that doesn't stand on its own

- "are" in [we're](#), French "je" in [j'ai](#), "le" in [l'honneur](#)

When should multiword expressions (MWE) be words?

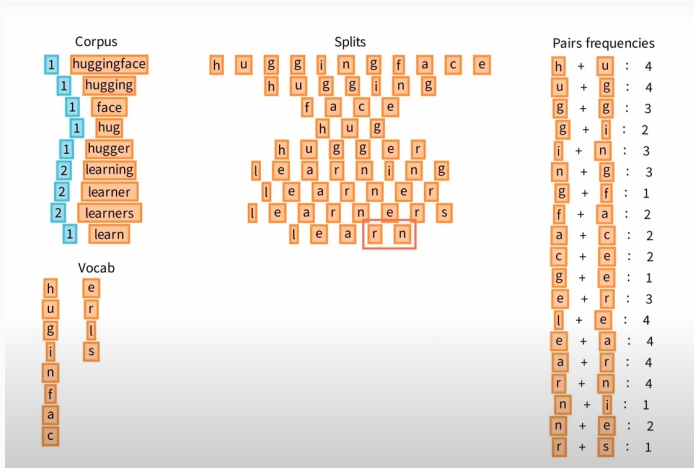
- [New York](#), [rock 'n' roll](#)

Tokenization in languages without spaces

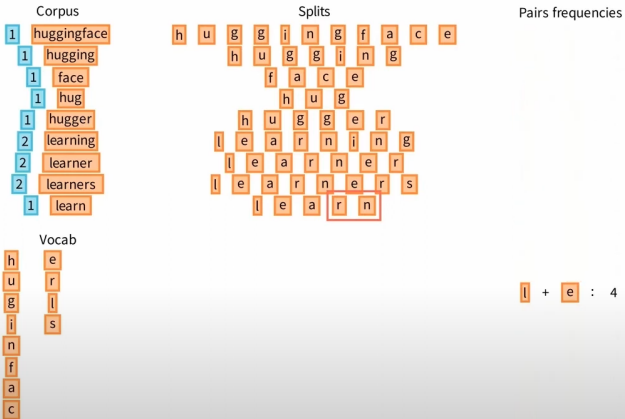
Many languages (like Chinese, Japanese, Thai) don't use spaces to separate words!

How do we decide where the token boundaries should be?

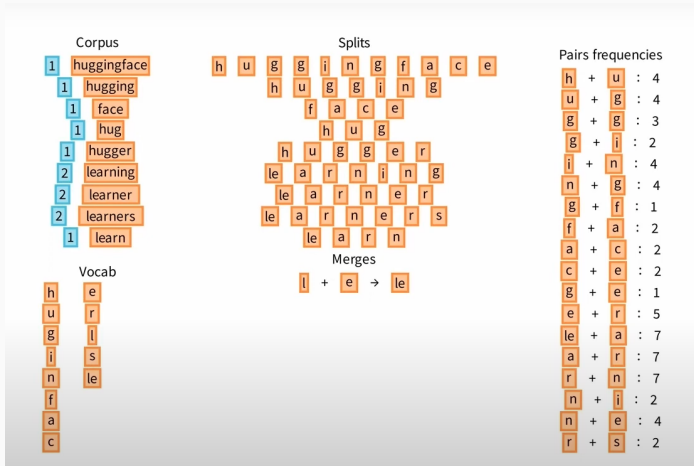
Byte Pair Encoding (BPE)



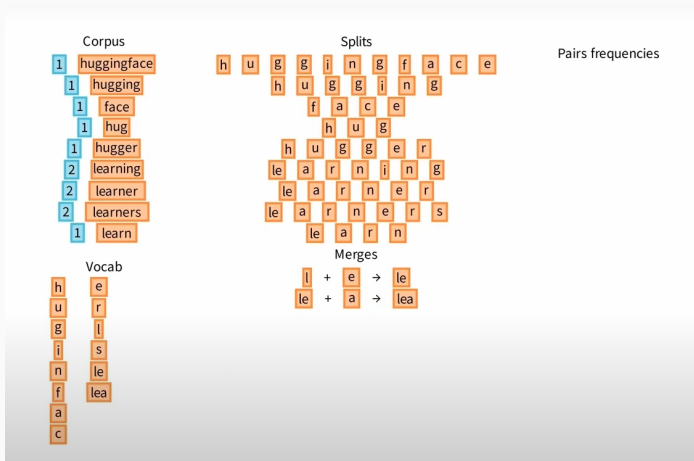
Byte Pair Encoding (BPE)



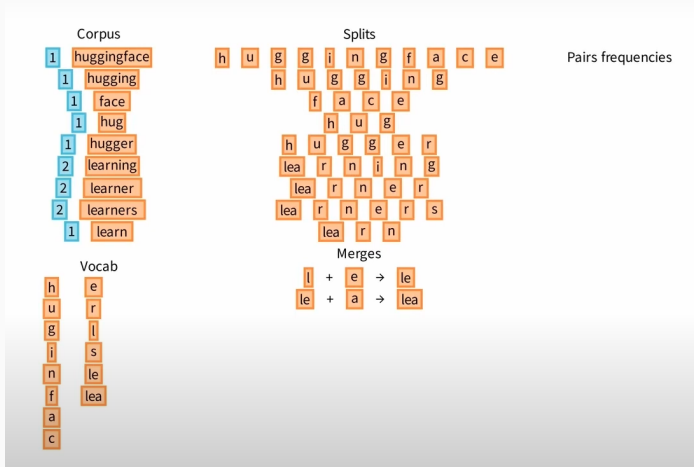
Byte Pair Encoding (BPE)



Byte Pair Encoding (BPE)



Byte Pair Encoding (BPE)



Byte Pair Encoding (BPE)

1. ("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
2. ("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)
3. Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug"]
Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)
4. Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un"]
Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("h" "ug" "s", 5)

Properties of BPE tokens

Usually include frequent words

And frequent subwords

- Which are often morphemes like *-est* or *-er*

A **morpheme** is the smallest meaning-bearing unit of a language

- *unlikeliest* has 3 morphemes *un-*, *likely*, and *-est*

Byte Pair Encoding (BPE)

1. Decoding : the encoded sequence [“the< /w >”, “high”, “est< /w >”, “range< /w >”, “in< /w >”, “Seattle< /w >”], we will be decoded as [“the”, “highest”, “range”, “in”, “Seattle”] and not as [“the”, “high”, “estranger”, “in”, “Seattle”]. Notice the presence of the “< /w >” token in “est”.
2. Encoding: we will iterate through all the tokens we found in our corpus — longest to the shortest and try to replace substrings in our given sequence of words using these tokens.

Word Normalization

Putting words/tokens in a standard format

- U.S.A. or USA
- uhhuh or uh-huh
- Fed or fed
- am, is, be, are

Case folding

Applications like IR: reduce all letters to lower case

- Since users tend to use lower case
- Possible exception: upper case in mid-sentence?
 - e.g., **General Motors**
 - **Fed** vs. *fed*
 - **SAIL** vs. *sail*

For sentiment analysis, MT, Information extraction

- Case is helpful (**US** versus *us* is important)

Lemmatization

Represent all words as their lemma, their shared root
= dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*
- Spanish **quiero** ('I want'), **quieres** ('you want')
→ **querer** 'want'
- *He is reading detective stories*
→ *He be read detective story*

Lemmatization is done by Morphological Parsing

Morphemes:

- The small meaningful units that make up words
- **Stems**: The core meaning-bearing units
- **Affixes**: Parts that adhere to stems, often with grammatical functions

Morphological Parsers:

- Parse *cats* into two morphemes *cat* and *s*

Stemming

Reduce terms to stems, chopping off affixes crudely

This was not the map we found in Billy Bones's chest, but an accurate copy, complete in all things-names and heights and soundings-with the single exception of the red crosses and the written notes.



Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note

.

Porter Stemmer

Based on a series of rewrite rules run in series

- A cascade, in which output of each pass fed to next pass

Some sample rules:

ATIONAL → ATE (e.g., relational → relate)

ING → ϵ if stem contains vowel (e.g., motoring → motor)

SSES → SS (e.g., grasses → grass)

Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...     ([A-Z]\.)+          # abbreviations, e.g. U.S.A.
...     | \w+(-\w+)*        # words with optional internal hyphens
...     | \$?\d+(\.\d+)?%?   # currency and percentages, e.g. $12.40, 82%
...     | \.\.\.           # ellipsis
...     | [][.,;'"'()?]-_[] # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```


Sentence Segmentation

!, ? mostly unambiguous but **period** “.” is very ambiguous

- Sentence boundary
- Abbreviations like Inc. or Dr.
- Numbers like .02% or 4.3

Common algorithm: Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary.

- An abbreviation dictionary can help

Sentence segmentation can then often be done by rules based on this tokenization.